Team Finatics — Day 1 to Day 44 Project Report

SentinelAI security intelligence and reporting project

1. Project Summary

This project, SentinelAI, is a Python-based security analysis and reporting system designed to convert raw security data into actionable vulnerability intelligence. The system maps Nmap output and event logs to OWASP Top 10:2025 and MITRE ATT&CK; techniques, adds AI-powered analysis when available, and generates structured remediation-focused reports.

The work spans from initial mapping logic and report design through testing, cross-platform validation, demo rehearsal, pull request finalization, and final Day 44 sprint completion.

2. Objective

The main goal was to build a workflow that accepts security observations, classifies them against standard frameworks, explains the risk in plain English, and produces a robust report that can be used by students, red teams, blue teams, and enterprise security reviewers.

The project was intentionally designed around authorized and in-scope usage, with consent and policy checks embedded to reduce misuse.

3. What Was Built

The repository contains the mapping engine, data loaders, report generator, remediation engine, and safety controls needed to support the full workflow.

Key modules include owasp_mapper.py, mitre_mapper.py, framework_mapper.py, nmap_parser.py, event_log_parser.py, llm_analyzer.py, remediation_mapper.py, report_generator.py, nmap_report_pipeline.py, and event_log_report_pipeline.py.

Safety checks were also added through consent_gate.py, domain_verifier.py, access_policy.py, session_guard.py, policy_guard.py, validation_guard.py, and compliance_guard.py.

4. Data Sources and Tools Used

The project relies on OWASP Top 10:2025 JSON data and MITRE ATT&CK; data files in the data/ directory. It uses Python for implementation, pytest for validation, and Google Gemini API for AI explanation when configured.

When the API is unavailable, the project falls back to a deterministic rule-based analyzer so testing remains offline and reliable. It is run inside a Python virtual environment and uses git, VS Code, PowerShell, and pip as the development workflow.

5. How the Workflow Works

1. Raw Nmap output or security event logs are fed into the parser layer.

2. Security indicators are extracted and deduplicated.

3. The framework mapper aligns each indicator to OWASP and MITRE categories.

4. AI analysis or fallback logic explains the risk in a human-readable way.

5. The report is generated in text, JSON, or Markdown format.

6. The result is returned as structured data plus final output for review.

6. Milestones Up to Day 44

Day 4: Report design and reporting structure work.

Day 15: End-to-end Nmap pipeline validation.

Day 16: Event log end-to-end validation.

Day 22: Code review and quality assessment.

Day 23: Full test run and validation milestone.

Day 26: Cross-platform testing confirmation on Windows and Linux.

Day 27: Demo rehearsal and presentation readiness.

Day 29: Pull request merge and repository completion step.

Day 30: Final sign-off and evidence-based verification.

Day 44: Sprint completion milestone for the authorized scan workflow.

7. Validation Evidence

The repository contains concrete verification documents, including Day30 final sign-off.md and the final sprint summary in SPRINT_DONE_DAY44.md. The project also includes pytest-based integration checks for the Nmap and event-log pipelines.

The recorded evidence states that the validation suite passed, the project was confirmed on both Windows and Linux, and the final sprint milestone was marked as complete.

8. Final Outcome

By Day 44, the project had reached a tested and documented state supporting OWASP mapping, MITRE mapping, remediation guidance, AI analysis, and multi-format reporting. The result is a functional security analysis workflow with governance and ethical-use boundaries built in.

This is a complete project lifecycle summary from design to validation and sign-off.